Robot EKF State Estimation

Robot EKF State Estimation

1. Course Content
2. Preparation
3. Case Demonstration
3. Viewing the Node Graph
4. Source Code Analysis

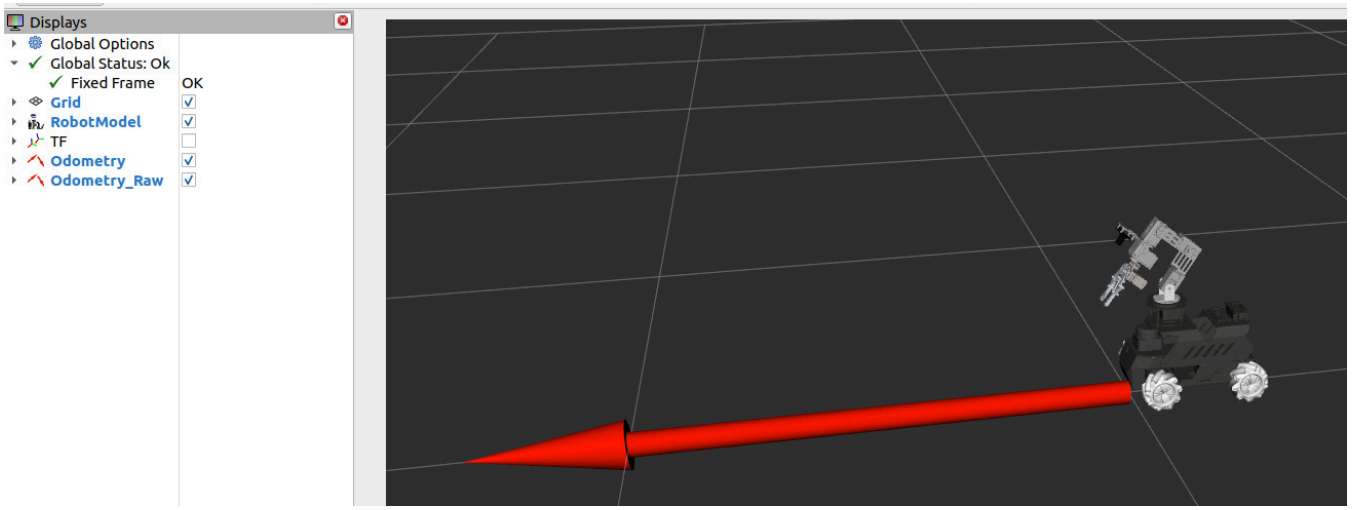
1. Course Content

- Perform multi-sensor data fusion using IMU and wheel encoder data to obtain fused odometry data odom and estimate the robot's movement position
- Observe in rviz the trajectory difference between the wheel odometry (blue arrow) and the odometry obtained after fusing IMU data (red arrow)

2. Preparation

- Start the state estimation node

```
ros2 launch yahboomcar_bringup state_estimation.launch.py
```



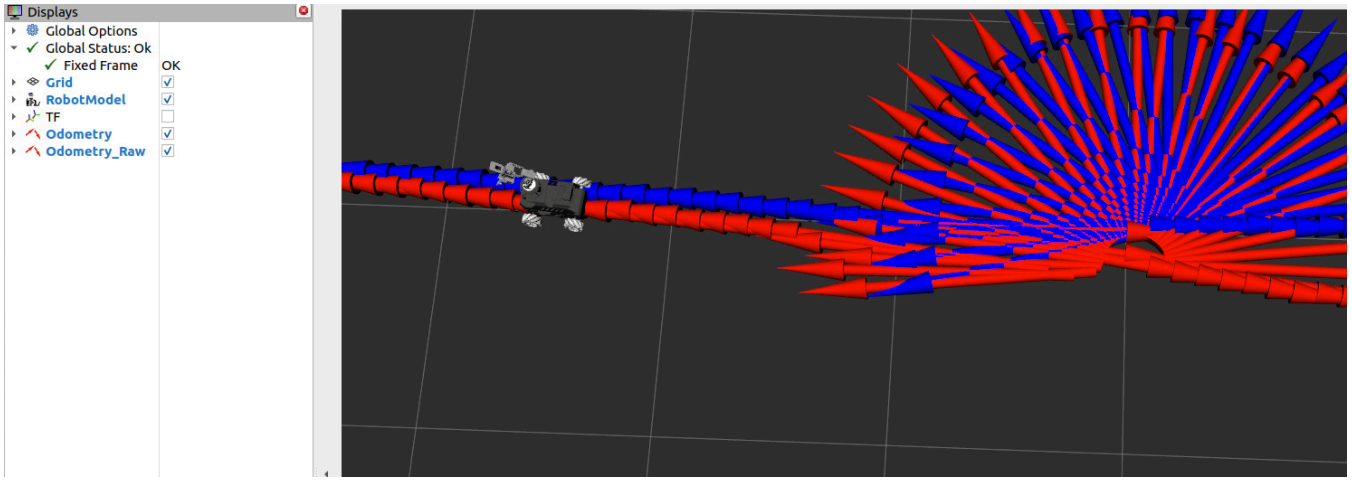
- Start the keyboard or handle control node; here, keyboard control is used as an example

```
ros2 run common_demo keyboard_control
```

3. Case Demonstration

- By controlling the car's movement with the keyboard, you can observe that the odometry trajectories of

odom (red arrow) and odom_raw (blue arrow) gradually diverge; EKF combines the wheel odometry and IMU data to obtain more stable odometry data

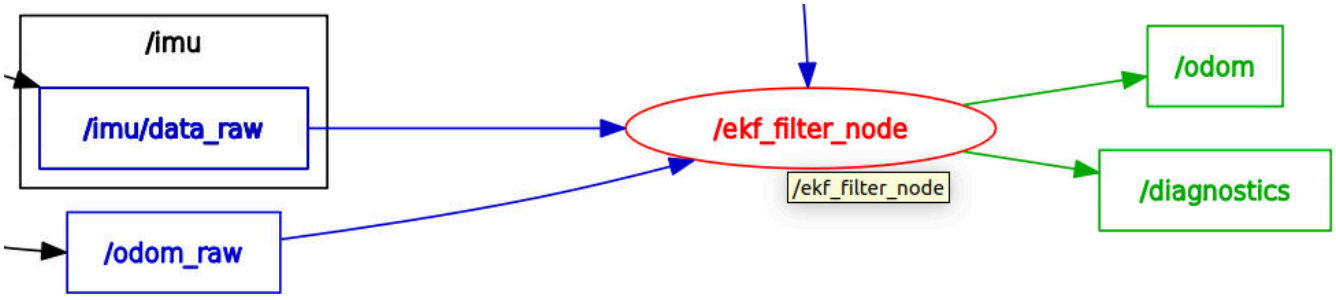


> **Important**
>
> In subsequent SLAM mapping and navigation-related functions, the odometry data used all comes from the EKF fused odometry as the data source

3. Viewing the Node Graph

```
rqt_graph
```

- If no node graph is displayed, click Refresh in the upper-left corner
- As can be seen, /odom_raw is the wheel odometry data calculated from the chassis motor encoder pulses, /imu/data_raw is the IMU data, and the ekf_filter_node node subscribes to them and publishes the fused odometry /odom



4. Source Code Analysis

- state_estimation launch file

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/launch/state_estimation.launch.py
```

`state_estimation.launch.py` adds rviz visualization on top of `car_base.launch.py`, used to compare the trajectory difference between the raw wheel odometry and the fused odometry:

1. `IncludeLaunchDescription`: reuses `car_base.launch.py`, passing in `use_camera:=false` to turn off the camera to save resources, and `log_level:=info` to enable detailed logging.
2. `rviz2`: loads the `state_estimation.rviz` configuration file, simultaneously displaying the two odometry trajectories `odom` (red) and `odom_raw` (blue), making it easy to visually compare the EKF fusion effect.

```
def generate_launch_description():
    rviz_config =
os.path.join(get_package_share_directory('yahboomcar_bringup'),'rviz','state_estimation.rviz
')
    carbase=IncludeLaunchDescription(

 PythonLaunchDescriptionSource(os.path.join(get_package_share_directory('yahboomcar_bringup'
),'launch','car_base.launch.py')),
            launch_arguments={'log_level': 'info',
                              'use_camera': 'false',
                              'use_lidar': 'false',
                              'odom_tf_source':'ekf'
                              }.items())

    rviz_node = Node(
        package = 'rviz2',
        executable = 'rviz2',
        name='rviz2',
        arguments=['-d', rviz_config,
                   '--ros-args', '--log-level', 'info'],
        output='screen'
    )

    return LaunchDescription([
        carbase,
        rviz_node
])
```

- ekf parameter file

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/ekf_M1.yaml
```

`ekf.yaml` is the core configuration of the EKF fusion node; the key parameters are described as follows:

- **Global parameters**: `frequency: 15.0` means the EKF runs at a frequency of 15Hz; `two_d_mode: true` restricts it to planar motion (ignoring z/roll/pitch); `publish_tf: true` automatically publishes the `odom → base_footprint` transform.
- `odom0: /odom_raw`: wheel odometry input. The 15-bit boolean array `odom0_config` corresponds to whether each of `[x,y,z, roll,pitch,yaw, vx,vy,vz, vroll,vpitch,vyaw, ax,ay,az]` participates in the fusion. Here only `vx` (forward velocity) and `yaw` (yaw angular velocity) are enabled, because a differential chassis can only reliably observe these two quantities.
- `imu0: /imu`: IMU input. `imu0_config` enables only `yaw` (yaw angle), using the IMU's

magnetometer/gyroscope to correct the wheel odometry's heading drift; `imu0_relative: true` means using the first IMU data as the reference baseline, avoiding jumps caused by inconsistency between the absolute heading and the odometry's initial orientation.

- **Fusion strategy**: the odometry provides translational velocity, and the IMU provides heading angle; the two are complementary — the odometry is accurate in the short term but its heading accumulates drift, while the IMU's heading is stable but has short-term noise. The EKF combines the advantages of both to obtain a more stable `/odom`.

```
ekf_filter_node:
    ros__parameters:
        use_sim_time: false
        frequency: 15.0
        sensor_timeout: 0.15
        two_d_mode: true
        transform_time_offset: 0.0
        transform_timeout: 0.05
        print_diagnostics: false
        debug: false
        publish_tf: true
        publish_acceleration: false
        reset_on_time_jump: true
        odom_frame: odom                # Defaults to "odom" if unspecified
        base_link_frame: base_footprint  # Defaults to "base_link" if unspecified
        world_frame: odom               # Defaults to the value of odom_frame if
unspecified

        odom0: /odom_raw
        odom0_config: [false, false, false,
                       false, false, false,
                       true, true, false,
                       false, false, true,
                       false, false, false]

        #         [x_pos   , y_pos    , z_pos,
        #          roll    , pitch    , yaw,
        #          x_vel   , y_vel    , z_vel,
        #          roll_vel, pitch_vel, yaw_vel,
        #          x_accel , y_accel  , z_accel]

        odom0_queue_size: 3
        odom0_nodelay: true
        odom0_differential: false
        odom0_relative: false
        odom0_pose_rejection_threshold: 20.0
        odom0_twist_rejection_threshold: 5.0

        imu0: /imu/data_raw
        imu0_config: [false, false, false,
                      false, false, true,
                      false, false, false,
                      false, false, false,
                      false, false, false]
```

```
        #        [x_pos   , y_pos    , z_pos,
        #         roll    , pitch    , yaw,
        #         x_vel   , y_vel    , z_vel,
        #         roll_vel, pitch_vel, yaw_vel,
        #         x_accel , y_accel  , z_accel]

        imu0_nodelay: true
        imu0_differential: false
        imu0_relative: true
        imu0_queue_size: 5
        imu0_pose_rejection_threshold: 5.0                # Note the difference in
parameter names     # 0.8
        imu0_twist_rejection_threshold: 5.0               # 0.8
        imu0_linear_acceleration_rejection_threshold: 10.0  #
        imu0_remove_gravitational_acceleration: true
```